

# Vasic-Digital-Reusable-Module-Suite

**Аднойчы стварыць, паўсюль выкарыстоўваць — флот невялікіх, развязаных, незалежна пратэставаных модуляў Go і KMP.**

## Зводка

Вялікае сямейства ўніверсальных, паўторна выкарыстоўваемых модуляў, апублікаваных пад прасторамі імёнаў `digital.vasic.*` (Go) і Kotlin Multiplatform. Кожны модуль з’яўляецца самастойным, незалежна пратэставаным і версіянаваным, і выкарыстоўваецца як падмодуль з аднолькавай кодовай базай у буйнейшых прадуктах (Catalogizer, HelixAgent і ўсёй астатняй флатыліі). Гэтая старонка аб’ядноўвае мноства невялікіх утыліт, якія былі б залішнімі як асобныя старонкі.

## Кароткае апісанне

Куратарскі набор развязаных модуляў `digital.vasic.*` — інфраструктурныя прымітывы (аўтэнтыфікацыя, кэш, база даных, канфігурацыя, назіральнасць), будаўнічыя блокі AI/агентаў (RAG, VectorDB, эмбедзінгі, MCP, агентыка, планаванне), а таксама абарончыя LLM-бар’еры (RedTeam, Normalize) — плюс дублікатны набор Kotlin Multiplatform. Кожны з іх з’яўляецца ўніверсальным, пратэставаным і паўторна выкарыстоўваемым.

## Падрабязнае апісанне

Арганізацыя vasic-digital будзецца на адной структурнай стаўцы: філасофіі «канстытуцыя + мноства развязаных паўторна выкарыстоўваемых падмодуляў», дзе ўніверсальны функцыянал ніколі не пішацца двойчы. Замест маналітаў кожная паўторна выкарыстоўваемая задача выносіцца ў асобны невялікі модуль — уласны рэпазіторый, уласныя тэсты,

уласная дакументацыя — і падтрымліваецца строга развязанай, каб ніякія спецыфікацыі спажыўца ніколі не прасочваліся ўнутр. Гэтая старонка аб'ядноўвае іх таму, што паасобку кожны з іх мае маштаб бібліятэкі і быў бы залішнім як асобная старонка прадукта. Разам жа яны ўяўляюць сабой сапраўдны сілавы множнік арганізацыі: прыватны інжынерны актыў, які ператварае «стварыць новы прадукт» у «сабраць з правяраных кампанентаў», і канкрэтнае абгрунтаванне таго, што гэтая флатылія не вынаходзіць кола нанова — яна падтрымлівае адно вельмі добрае кола і каціць яго паўсюль.

Набор ахоплівае тры кластары. **Інфраструктурныя прымітывы** (Go) забяспечваюць асноўную інфраструктуру, неабходную кожнаму сэрвісу: auth (JWT/bcrypt), cache (Redis/TTL), database (міграцыі, дуальныя SQLite/PostgreSQL), config, middleware, observability (Prometheus/OpenTelemetry), ratelimiter, security, storage (S3/MinIO), streaming (WebSocket-хаб), eventbus, filesystem (мультипраатакол), discovery/mdns, http3, recovery, concurrency, lazy і іншыя. **Будаўнічыя блокі AI/агентаў** (Go) забяспечваюць аснову для сістэм AI: rag, vectordb, embeddings, memory, conversation (сцісканне бясконцага кантэксту, event sourcing), mcp (Model Context Protocol), toolschema, skillregistry, agentic (аркестроўка працоўных працэсаў на аснове графаў), planning (HiPlan/MCTS/Tree-of-Thoughts), benchmark (SWE-bench/HumanEval/MMLU), llmops, selfimprove (мадэляванне ўзнагарод/RLHF) і toon (Token-Oriented Object Notation). **Абарончыя LLM-бар'еры** забяспечваюць інструменты супрацьстаяння адверсарыяльным пагрозам: RedTeam (адверсарыяльныя фікстуры на аснове YAML), Normalize (кананізацыя адверсарыяльных уваходаў). Паралельны набор **Kotlin Multiplatform** дублюе асноўныя модулі (Auth-KMP, Database-KMP, Security-KMP, UI-Components-KMP і г.д.) для кросплатформавых праграм.

Змест

## Чаму мы гэта стварылі

Запуск шматлікіх прадуктаў (Catalogizer, HelixAgent, Herald і іншых) з нуля кожны раз — марнатраўства рэсурсаў і непаслядоўнасць. Вынясенне ўсіх агульных задач у асобныя, дэкапліраваныя і пратэставаныя модулі азначае, што выпраўленні і паляпшэнні распаўсюджваюцца на ўсю сістэму, а кожны новы прадукт збіраецца з правяраных кампанентаў.

# Чаму гэта змяняе правілы гульні

Гэта, па сутнасці, прыватная «стандартная бібліятэка» для стварэння бэкэндаў, засяроджаных на AI — той пласт, які большасць каманд ніколі не паспявае пабудаваць, бо яны занадта занятыя паўторным вырашэннем пытанняў аўтэнтыфікацыі, кэшавання і інфраструктурных дэталей RAG у пяты раз. Тут інфраструктурныя прымітывы, будаўнічыя блокі AI і абарончыя агароджы LLM існуюць як гатовыя да ўжывання, незалежна пратэставаныя модулі. Гэта дазваляе невялікай камандзе ствараць сістэмы прадукцыйнага ўзроўню з такой хуткасцю, якая звычайна патрабуе значна большых рэсурсаў, і рабіць гэта без дублявання, якое звычайна назапашваецца ў працэсе.

## Што новага

- **Дысцыпліна дэкаплінгу на ўзроўні ўсёй сістэмы (CONST-051):** субмодулі разглядаюцца як самастойныя кодовыя базы, якія ніколі не ўтрымліваюць спецыфіку спажывацёў.
- **Асобны пласт прымітываў AI** (RAG, VectorDB, Embeddings, MCP, ToolSchema, Agentic, Planning, LLMOps) як паўторна выкарыстоўваемыя модулі.
- **Кластар абарончых агароджаў LLM** (RedTeam, Normalize) для супрацьстаяння адверскім атакам.
- **Паралельныя наборы модуляў Go і Kotlin Multiplatform** з аднолькавымі канвенцыямі.

## Праблемы і рашэнні

- **Пазбяганне дэградацыі дэкаплінгу паміж дзясяткамі модуляў:** вырашана праз кантракт дэкаплінгу ў канстытуцыі і ін'екцыю спецыфікі спажывацёў падчас выканання.
- **Падтрыманне аднастайнасці і праверанай якасці шматлікіх модуляў:** вырашана праз агульную канвенцыю (тэсты/дакументацыя/Challenge для кожнага модуля) і кіраўніцкую аснову HelixConstitution.
- **Ахоп розных платформаў:** вырашана праз люстэркаванне асноўных модуляў на Kotlin Multiplatform.

## Тэхнічны стэк (чаму і як)

- **Go** — большасць модуляў (digital.vasic.\*).

- **Kotlin Multiplatform** — кроссплатформавыя люстэркаваныя модулі (Auth/Database/Security/UI/Concurrency/RateLimiter-KMP).
- **Redis / PostgreSQL / SQLite** — прымітывы кэша, баз даных і сховішчаў.
- **Prometheus / OpenTelemetry** — модуль назіральнасці.
- **WebSocket / HTTP/3 (quic-go) / mDNS** — сеткавыя модулі.
- **Vector DB / embeddings / RAG / MCP** — модулі прымітываў AI.
- **YAML** — адверскія фікстуры і канфігурацыі RedTeam.

НЕПРАВЕРАНА / У РАСПРАЦОЎЦЫ: некалькі арганізацыйных рэпазіторыяў пазначаны як «SCAFFOLD / WIP» (напрыклад, PliniusCommon, I-LLM, HyperTune, AutoTemp, Veritas, Ouroborous, Claritas, LeakHub, GandalfSolutions). Прадстаўляйце іх як раннія/каркасныя версіі, якія яшчэ не выпушчаны.